# 9-Month Daily Development Plan - Women-Only Ride Hailing Platform

This roadmap is designed for a solo fullstack developer building a production-level women-only ride-hailing platform with advanced safety systems, GPS tracking, real-time communication, temporary dashcam evidence storage, and full DevOps infrastructure.

Tech Stack: React Native + TypeScript - Node.js + Express - PostgreSQL - Prisma ORM - Socket.IO - Google Maps APIs - Paystack - Docker - Kubernetes - GitHub Actions

## Month 1 - Backend Foundation and Architecture

- Day 1: Setup monorepo/project structure
- Day 2: Setup monorepo/project structure
- Day 3: Configure PostgreSQL and Prisma
- Day 4: Configure PostgreSQL and Prisma
- Day 5: Design database schema
- Day 6: Design database schema
- Day 7: Setup Express server architecture
- Day 8: Setup Express server architecture
- Day 9: Implement JWT authentication
- Day 10: Implement JWT authentication
- Day 11: Create rider and driver roles
- Day 12: Create rider and driver roles
- Day 13: Build OTP verification flow
- Day 14: Build OTP verification flow
- Day 15: Setup environment configuration
- Day 16: Setup environment configuration
- Day 17: Implement logging and error handling
- Day 18: Implement logging and error handling
- Day 19: Create base API documentation
- Day 20: Create base API documentation
- Day 21: Setup Docker and deployment pipeline
- Day 22: Setup Docker and deployment pipeline
- Day 23: Setup Git workflow and branches
- Day 24: Setup Git workflow and branches

- Day 25: Implement user profile APIs
- Day 26: Implement user profile APIs
- Day 27: Implement vehicle registration APIs
- Day 28: Implement vehicle registration APIs
- Day 29: Add validation middleware (Zod)
- Day 30: Add validation middleware (Zod)
- Day 31: Write authentication tests
- Day 32: Write authentication tests
- Day 33: Review and refactor architecture
- Day 34: Review and refactor architecture
- Day 35: Buffer/debugging days
- Day 36: Buffer/debugging days

## Month 2 - Ride Lifecycle System

- Day 1: Build ride request flow
- Day 2: Build ride request flow
- Day 3: Implement ride status lifecycle
- Day 4: Implement ride status lifecycle
- Day 5: Setup Socket.IO server
- Day 6: Setup Socket.IO server
- Day 7: Implement real-time ride updates
- Day 8: Implement real-time ride updates
- Day 9: Build driver availability system - driver becomes unavailable when on active ride
- Day 10: Build driver availability system - driver becomes unavailable when on active ride
- Day 11: Implement nearby driver matching - 5km radius using Haversine formula
- Day 12: Implement nearby driver matching - 5km radius using Haversine formula
- Day 13: Create ride cancellation system
- Day 14: Create ride cancellation system
- Day 15: Build PricingConfig table and seed Lagos default pricing (NGN 1000 base, NGN 250/km, 70% floor)
- Day 16: Build PricingConfig table and seed Lagos default pricing (NGN 1000 base, NGN 250/km, 70% floor)
- Day 17: Build fareCalculator utility - fetch config from DB, calculate suggested fare and floor price
- Day 18: Build fareCalculator utility - fetch config from DB, calculate suggested fare and floor price

- Day 19: Implement InDrive fare model - rider proposes fare above floor, driver accepts or counter-offers
- Day 20: Implement InDrive fare model - rider proposes fare above floor, driver accepts or counter-offers
- Day 21: Build fare negotiation socket events - accept_offer, counter_offer, accept_counter, reject_counter
- Day 22: Build fare negotiation socket events - accept_offer, counter_offer, accept_counter, reject_counter
- Day 23: Build ride confirmation API - send driver name, car info, plate, phone and agreed fare to rider
- Day 24: Build ride confirmation API - send driver name, car info, plate, phone and agreed fare to rider
- Day 25: Emit ride_accepted socket event with driver details and agreed fare to rider
- Day 26: Emit ride_assigned socket event with rider details and agreed fare to driver
- Day 27: Implement no drivers found flow - expand search 5km to 7km to 10km over 90 seconds
- Day 28: Implement no drivers found flow - expand search 5km to 7km to 10km over 90 seconds
- Day 29: Handle reconnect scenarios
- Day 30: Handle reconnect scenarios
- Day 31: Test real-time synchronization
- Day 32: Test real-time synchronization
- Day 33: Write backend tests
- Day 34: Write backend tests
- Day 35: Buffer/debugging days
- Day 36: Buffer/debugging days

## Month 3 - Rider Mobile App

- Day 1: Setup React Native project
- Day 2: Setup React Native project
- Day 3: Configure navigation
- Day 4: Configure navigation
- Day 5: Implement authentication screens
- Day 6: Implement authentication screens
- Day 7: Create onboarding flow
- Day 8: Create onboarding flow
- Day 9: Build home/map screen

- Day 10: Build home/map screen
- Day 11: Integrate Google Maps
- Day 12: Integrate Google Maps
- Day 13: Build ride booking UI - show system suggested fare and minimum offer to rider
- Day 14: Build ride booking UI - show system suggested fare and minimum offer to rider
- Day 15: Build fare input UI - rider can increase fare but blocked from going below floor price
- Day 16: Build fare input UI - rider can increase fare but blocked from going below floor price
- Day 17: Build fare negotiation UI - show driver counter-offer with accept and reject buttons
- Day 18: Build fare negotiation UI - show driver counter-offer with accept and reject buttons
- Day 19: Create live tracking screen
- Day 20: Create live tracking screen
- Day 21: Build ride confirmation screen - show driver name, photo, car, plate, phone and agreed fare
- Day 22: Build ride confirmation screen - show driver name, photo, car, plate, phone and agreed fare
- Day 23: Build searching for driver screen - show real-time status as search radius expands
- Day 24: Build searching for driver screen - show real-time status as search radius expands
- Day 25: Add trip history screen
- Day 26: Add trip history screen
- Day 27: Build user profile screens
- Day 28: Build user profile screens
- Day 29: Implement API integration layer
- Day 30: Implement API integration layer
- Day 31: Setup state management
- Day 32: Setup state management

## Month 4 - Driver Mobile App

- Day 1: Build driver authentication flow
- Day 2: Build driver authentication flow
- Day 3: Create driver dashboard
- Day 4: Create driver dashboard
- Day 5: Implement online/offline toggle
- Day 6: Implement online/offline toggle
- Day 7: Build ride offer screen - show rider proposed fare with accept and counter-offer buttons

- Day 8: Build ride offer screen - show rider proposed fare with accept and counter-offer buttons
- Day 9: Build counter-offer UI - driver enters preferred fare and submits
- Day 10: Build counter-offer UI - driver enters preferred fare and submits
- Day 11: Build ride accepted screen - show rider name, phone and agreed fare
- Day 12: Build ride accepted screen - show rider name, phone and agreed fare
- Day 13: Create navigation integration
- Day 14: Create navigation integration
- Day 15: Build earnings screen
- Day 16: Build earnings screen
- Day 17: Implement real-time driver updates
- Day 18: Implement real-time driver updates
- Day 19: Add trip completion system
- Day 20: Add trip completion system
- Day 21: Integrate push notifications
- Day 22: Integrate push notifications
- Day 23: Optimize battery usage
- Day 24: Optimize battery usage
- Day 25: Handle background location tracking
- Day 26: Handle background location tracking
- Day 27: Testing and refactoring
- Day 28: Testing and refactoring
- Day 29: Buffer/debugging days
- Day 30: Buffer/debugging days

## Month 5 - Payments and Admin Dashboard

- Day 1: Integrate Paystack
- Day 2: Integrate Paystack
- Day 3: Implement wallet system
- Day 4: Implement wallet system
- Day 5: Create payment APIs - charge rider based on agreedFare after trip completes
- Day 6: Create payment APIs - charge rider based on agreedFare after trip completes
- Day 7: Build admin authentication
- Day 8: Build admin authentication
- Day 9: Create admin dashboard UI

- Day 10: Create admin dashboard UI
- Day 11: Build user management system
- Day 12: Build user management system
- Day 13: Implement ride monitoring tools
- Day 14: Implement ride monitoring tools
- Day 15: Build analytics overview - include average offered vs agreed fare data
- Day 16: Build analytics overview - include average offered vs agreed fare data
- Day 17: Build pricing management UI - admin can update PricingConfig without code changes
- Day 18: Build pricing management UI - admin can update PricingConfig without code changes
- Day 19: Add dispute management
- Day 20: Add dispute management
- Day 21: Implement driver verification workflow
- Day 22: Implement driver verification workflow
- Day 23: Write payment tests
- Day 24: Write payment tests
- Day 25: Refactor dashboard architecture
- Day 26: Refactor dashboard architecture
- Day 27: Buffer/testing days
- Day 28: Buffer/testing days

## Month 6 - Safety Systems

- Day 1: Implement SOS system
- Day 2: Implement SOS system
- Day 3: Build emergency contact flow
- Day 4: Build emergency contact flow
- Day 5: Create trip sharing feature
- Day 6: Create trip sharing feature
- Day 7: Implement route deviation detection - alert rider, admin and emergency contacts
- Day 8: Implement route deviation detection - alert rider, admin and emergency contacts
- Day 9: Add suspicious activity alerts
- Day 10: Add suspicious activity alerts
- Day 11: Build incident reporting APIs
- Day 12: Build incident reporting APIs
- Day 13: Create emergency admin panel

- Day 14: Create emergency admin panel
- Day 15: Integrate push emergency alerts
- Day 16: Integrate push emergency alerts
- Day 17: Optimize real-time safety updates
- Day 18: Optimize real-time safety updates
- Day 19: Security review
- Day 20: Security review
- Day 21: Penetration testing basics
- Day 22: Penetration testing basics
- Day 23: Improve app reliability
- Day 24: Improve app reliability
- Day 25: Buffer/debugging days
- Day 26: Buffer/debugging days

## Month 7 - Dashcam and Evidence System

- Day 1: Research dashcam integration
- Day 2: Research dashcam integration
- Day 3: Design video upload architecture
- Day 4: Design video upload architecture
- Day 5: Implement encrypted chunk uploads
- Day 6: Implement encrypted chunk uploads
- Day 7: Setup temporary cloud storage
- Day 8: Setup temporary cloud storage
- Day 9: Create auto-deletion jobs
- Day 10: Create auto-deletion jobs
- Day 11: Implement evidence locking
- Day 12: Implement evidence locking
- Day 13: Build incident-based preservation system
- Day 14: Build incident-based preservation system
- Day 15: Implement secure video access
- Day 16: Implement secure video access
- Day 17: Add access logging
- Day 18: Add access logging
- Day 19: Optimize storage costs

- Day 20: Optimize storage costs
- Day 21: Handle failed uploads
- Day 22: Handle failed uploads
- Day 23: Test edge cases
- Day 24: Test edge cases
- Day 25: Security testing
- Day 26: Security testing
- Day 27: Buffer/debugging days
- Day 28: Buffer/debugging days

## Month 8 - Final App Testing and Polish

- Day 1: Full application testing - backend, rider app and driver app
- Day 2: Full application testing - backend, rider app and driver app
- Day 3: Fix critical bugs
- Day 4: Fix critical bugs
- Day 5: Optimize database queries
- Day 6: Optimize database queries
- Day 7: Improve socket performance
- Day 8: Improve socket performance
- Day 9: Optimize mobile UX
- Day 10: Optimize mobile UX
- Day 11: Stress test backend
- Day 12: Stress test backend
- Day 13: Implement monitoring/logging
- Day 14: Implement monitoring/logging
- Day 15: Setup database backups
- Day 16: Setup database backups
- Day 17: Prepare launch checklist
- Day 18: Prepare launch checklist
- Day 19: Prepare onboarding process for riders and drivers
- Day 20: Prepare onboarding process for riders and drivers
- Day 21: Create support workflows
- Day 22: Create support workflows
- Day 23: Final refactoring

- Day 24: Final refactoring
- Day 25: Buffer/debugging days
- Day 26: Buffer/debugging days

## Month 9 - DevOps and Infrastructure

Full DevOps setup including containerization, CI/CD pipeline, Kubernetes orchestration, monitoring and production launch.

- Day 1: Write Dockerfile for Node.js backend - optimize for production
- Day 2: Write Dockerfile for Node.js backend - optimize for production
- Day 3: Write docker-compose.yml for local development (app + postgres + redis in one command)
- Day 4: Write docker-compose.yml for local development (app + postgres + redis in one command)
- Day 5: Write .dockerignore and optimize Docker image size
- Day 6: Write .dockerignore and optimize Docker image size
- Day 7: Test fully containerized app locally with docker-compose
- Day 8: Test fully containerized app locally with docker-compose
- Day 9: Setup GitHub Actions workflow file (.github/workflows/deploy.yml)
- Day 10: Setup GitHub Actions workflow file (.github/workflows/deploy.yml)
- Day 11: Build CI pipeline - on push: run tests, lint code, build Docker image
- Day 12: Build CI pipeline - on push: run tests, lint code, build Docker image
- Day 13: Build CD pipeline - on tests pass: push Docker image to GitHub Container Registry
- Day 14: Build CD pipeline - on tests pass: push Docker image to GitHub Container Registry
- Day 15: Setup staging environment - auto deploy to staging on every push to main branch
- Day 16: Setup staging environment - auto deploy to staging on every push to main branch
- Day 17: Setup production environment - manual approval required before deploying to production
- Day 18: Setup production environment - manual approval required before deploying to production
- Day 19: Learn Kubernetes core concepts - pods, deployments, services, ingress, namespaces
- Day 20: Learn Kubernetes core concepts - pods, deployments, services, ingress, namespaces
- Day 21: Write Kubernetes deployment manifest for backend (replicas, resource limits, health checks)
- Day 22: Write Kubernetes deployment manifest for backend (replicas, resource limits, health checks)
- Day 23: Write Kubernetes service and ingress config - route traffic to backend pods

- Day 24: Write Kubernetes service and ingress config - route traffic to backend pods
- Day 25: Setup Helm charts - package all Kubernetes manifests for easier deployment management
- Day 26: Setup Helm charts - package all Kubernetes manifests for easier deployment management
- Day 27: Configure Kubernetes auto-scaling (HPA) - scale pods based on CPU and memory usage
- Day 28: Configure Kubernetes auto-scaling (HPA) - scale pods based on CPU and memory usage
- Day 29: Setup Prometheus for metrics collection from backend and Kubernetes cluster
- Day 30: Setup Prometheus for metrics collection from backend and Kubernetes cluster
- Day 31: Setup Grafana dashboards - visualize API response times, error rates, active rides
- Day 32: Setup Grafana dashboards - visualize API response times, error rates, active rides
- Day 33: Setup centralized logging - collect logs from all pods in one place
- Day 34: Setup centralized logging - collect logs from all pods in one place
- Day 35: Setup alerting - get notified when server is down, error rate spikes or memory is high
- Day 36: Setup alerting - get notified when server is down, error rate spikes or memory is high
- Day 37: Configure SSL/HTTPS certificates for production domain
- Day 38: Configure SSL/HTTPS certificates for production domain
- Day 39: Load test production infrastructure - simulate hundreds of concurrent users
- Day 40: Load test production infrastructure - simulate hundreds of concurrent users
- Day 41: Final production launch - go live
- Day 42: Final production launch - go live and monitor closely for first 48 hours

## Important Notes

- Focus on shipping features before perfect optimization.
- Test continuously instead of postponing all testing.
- Keep architecture modular but avoid overengineering.
- Use AI tools aggressively to speed up development.
- Launching a stable product matters more than building every advanced feature immediately.
- Phone numbers are shown to matched rider/driver only - never exposed publicly.
- Driver eligibility is always verified server-side - never trust the client.
- Route deviation alerts go to rider, admin AND emergency contacts simultaneously.
- No drivers found: expand search from 5km to 7km to 10km over 90 seconds before giving up.
- Fare floor enforced server-side - rider cannot submit offer below 70% of suggested fare.

- Payment at trip completion is always based on agreedFare - never offeredFare or counterFare.
- Pricing config lives in database - admin changes prices without redeploying code.
- All Docker images go through GitHub Actions CI/CD before reaching production.
- Kubernetes handles zero-downtime deployments - users never see downtime during updates.